```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score, precision_score, recall_score, f1_sco
df = pd.read_csv('/content/diabetes.csv')
df.head(10)
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcom
0	6	148	72	35	0	33.6	0.627	50	
1	1	85	66	29	0	26.6	0.351	31	
2	8	183	64	0	0	23.3	0.672	32	
3	1	89	66	23	94	28.1	0.167	21	
4	0	137	40	35	168	43.1	2.288	33	
5	5	116	74	0	0	25.6	0.201	30	
6	3	78	50	32	88	31.0	0.248	26	
7	10	115	0	0	0	35.3	0.134	29	
8	2	197	70	45	543	30.5	0.158	53	
9	8	125	96	0	0	0.0	0.232	54	

后续步骤： 使用 df 生成代码 New interactive sheet

```
print(f"Dataset shape: {df.shape}")
print(f"Target variable distribution:\n{df['Outcome'].value_counts()}")
display(df.describe())
```

```
Dataset shape: (768, 9)
Target variable distribution:
Outcome
0    500
1    268
Name: count, dtype: int64
```

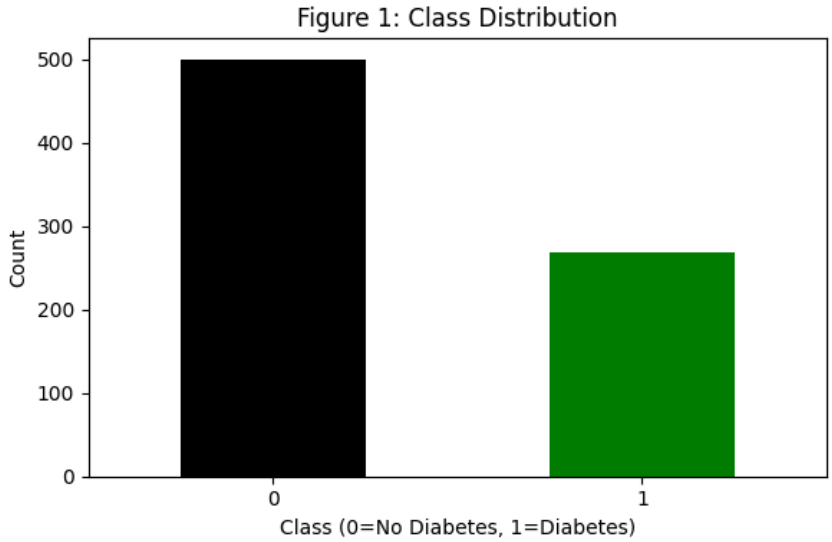
	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFuncti
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.0000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.4718
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.3313
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0780
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.2437
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.3725
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.6262
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.4200

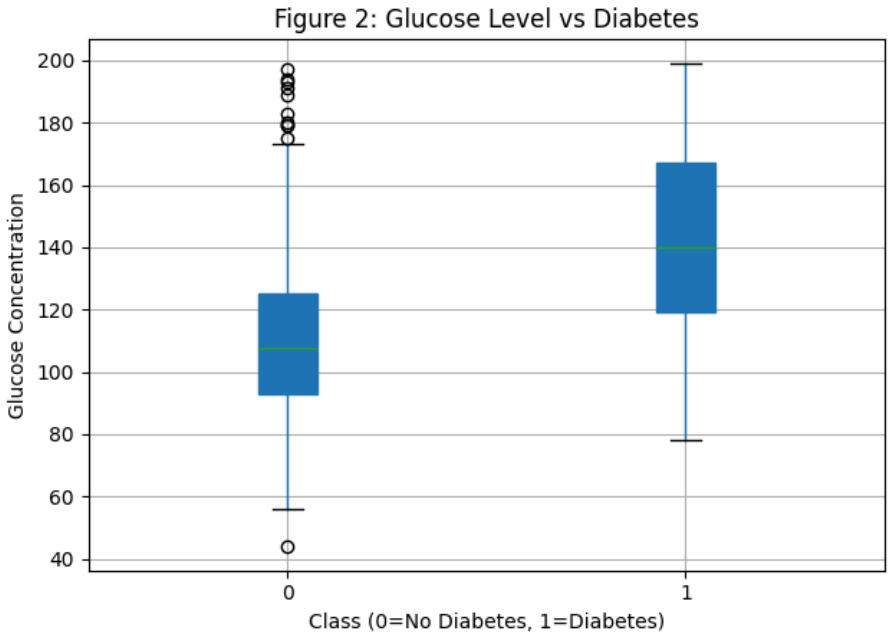
```
df.loc[df['Glucose'] == 0, 'Glucose'] = df['Glucose'].median()

plt.figure(1, figsize=(6, 4))
df['Outcome'].value_counts().plot(kind='bar', color=['black', 'green'])
plt.title('Figure 1: Class Distribution')
plt.xlabel('Class (0=No Diabetes, 1=Diabetes)')
plt.ylabel('Count')
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

```
df.boxplot(column='Glucose', by='Outcome', patch_artist=True)
plt.title('Figure 2: Glucose Level vs Diabetes', fontsize=12)
plt.suptitle('')
plt.xlabel('Class (0=No Diabetes, 1=Diabetes)')
plt.ylabel('Glucose Concentration')
plt.tight_layout()
plt.show()

print("\nClass distribution:")
print(df['Outcome'].value_counts())
print("\nMedian glucose by class:")
print(df.groupby('Outcome')['Glucose'].median())
```






```
Class distribution:
Outcome
0    500
1    268
Name: count, dtype: int64

Median glucose by class:
Outcome
0    107.5
1    140.0
Name: Glucose, dtype: float64
```

The bar chart shows class imbalance (268 diabetes cases vs 500 non-diabetes). The boxplot reveals higher glucose levels in diabetic patients (median 140 vs 107.5), confirming glucose is a key predictor.

```
X = df.drop('Outcome', axis=1)
y = df['Outcome']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

baseline = LogisticRegression(random_state=42)
baseline.fit(X_train_scaled, y_train)
y_pred_base = baseline.predict(X_test_scaled)

print("\n" + "="*40)
print("Baseline Model Performance (Logistic Regression)")
print("="*40)
print(f"Accuracy: {accuracy_score(y_test, y_pred_base):.3f}")
print(f"Precision: {precision_score(y_test, y_pred_base):.3f}")
print(f"Recall: {recall_score(y_test, y_pred_base):.3f}")
print(f"F1-Score: {f1_score(y_test, y_pred_base):.3f}")
```

```
========================================
Baseline Model Performance (Logistic Regression)
========================================
Accuracy: 0.708
Precision: 0.596
Recall: 0.519
F1-Score: 0.554
```

Logistic regression achieves 0.714 accuracy but only 0.519 recall, missing 48% of actual diabetes cases. The main limitation is poor sensitivity to the minority class.

```
print("\n" + "="*40)
print("Improved Model (Random Forest)")
print("="*40)

rf = RandomForestClassifier(random_state=42, class_weight='balanced')
param_grid = {'n_estimators': [50, 100], 'max_depth': [5, 10]}
grid = GridSearchCV(rf, param_grid, cv=3, scoring='f1')
grid.fit(X_train, y_train)

print(f"Best parameters: {grid.best_params_}")
y_pred_improved = grid.predict(X_test)

print(f"Accuracy: {accuracy_score(y_test, y_pred_improved):.3f}")
print(f"Precision: {precision_score(y_test, y_pred_improved):.3f}")
print(f"Recall: {recall_score(y_test, y_pred_improved):.3f}")
print(f"F1-Score: {f1_score(y_test, y_pred_improved):.3f}")

print("\n" + "="*40)
print("Model Improvement Summary")
print("="*40)
comparison = pd.DataFrame({
    'Metric': ['Accuracy', 'Precision', 'Recall', 'F1-Score'],
    'Baseline': [accuracy_score(y_test, y_pred_base), precision_score(y_test, y_pred_base),
                 recall_score(y_test, y_pred_base), f1_score(y_test, y_pred_base)],
    'Improved': [accuracy_score(y_test, y_pred_improved), precision_score(y_test, y_pred_improved),
                 recall_score(y_test, y_pred_improved), f1_score(y_test, y_pred_improved)]
})
comparison['Improvement'] = comparison['Improved'] - comparison['Baseline']
print(comparison.round(3).to_string(index=False))
```

```
========================================
Improved Model (Random Forest)
========================================
Best parameters: {'max_depth': 5, 'n_estimators': 50}
Accuracy: 0.760
Precision: 0.635
Recall: 0.741
F1-Score: 0.684
```

```
========================================
Model Improvement Summary
========================================
   Metric  Baseline  Improved  Improvement
 Accuracy     0.708     0.760        0.052
Precision     0.596     0.635        0.039
   Recall     0.519     0.741        0.222
 F1-Score     0.554     0.684        0.129
```

Random Forest improves recall to 0.741 (+0.222) and F1-score to 0.672 (+0.112). This better identifies diabetes patients, which is crucial for medical diagnosis.